

Day 9: Java 8 Features (18-June-2026, Thursday)

Session Time: 9:00 AM – 4:30 PM (Lunch Break: 12:30 PM – 1:30 PM, Tea Breaks: 10:30 AM & 3:00 PM)

Session Objectives

By the end of Day 9, learners will be able to:

- Write concise and expressive code using **Lambda Expressions**.
- Recognize and use built-in **Functional Interfaces** (`Predicate` , `Consumer` , `Function` , `Supplier`).
- Process collections declaratively using the **Stream API** (filter, map, reduce, collect).
- Handle potentially absent values safely with the **Optional** class.
- Use **Method References** to simplify lambda expressions.
- Solve LeetCode: *Merge Intervals* and *Binary Search* (applying functional thinking).
- Solve HackerRank: *Java Lambda Expressions* (custom functional interface).
- Design a **Real-Time Data Analytics Pipeline** using Java 8 features.

Detailed Agenda (Time Slots & Activities)

Time Slot	Activity	Duration
9:00 – 9:15	Recap of Day 8 & Day 9 Overview – Java 8 Revolution	15 min
9:15 – 10:00	Lambda Expressions – Syntax, Use Cases, Variable Capture	45 min
10:00 – 10:45	Functional Interfaces – Predicate, Consumer, Function, Supplier	45 min
10:45 – 11:00	Tea Break	15 min
11:00 – 11:45	Stream API – Creation, Intermediate Operations (filter, map, sorted)	45 min
11:45 – 12:15	Stream API – Terminal Operations (collect, forEach, reduce, count)	30 min

12:15 – 12:30	Hands-on: Stream Pipeline Exercises	15 min
12:30 – 1:30	Lunch Break	60 min
1:30 – 2:00	Optional Class – Creation, Methods, Safe Handling	30 min
2:00 – 2:30	Method References – Static, Instance, Constructor Reference	30 min
2:30 – 2:45	HackerRank: Java Lambda Expressions Walkthrough	15 min
2:45 – 3:00	Tea Break	15 min
3:00 – 3:30	LeetCode: Merge Intervals (sorting + lambda)	30 min
3:30 – 4:00	LeetCode: Binary Search (iterative + functional)	30 min
4:00 – 4:30	Corporate Use Case: Real-Time Data Analytics Pipeline	30 min

1. Lambda Expressions

1.1 What are Lambda Expressions?

- A lambda expression is a **concise way to represent an anonymous function** (a function without a name).
- Enables **functional programming** in Java – treat functionality as method argument.
- Syntactic sugar over **anonymous inner classes** (especially for functional interfaces).

1.2 Syntax

```
(parameters) -> { body }
```

- **Parameters:** Can have zero or more parameters. Type can be inferred.
- **Arrow token:** `->` separates parameters from body.
- **Body:** Can be a single expression (no `{ }` and implicit return) or a block (with `{ }` and explicit return).

1.3 Examples

Lambda	Equivalent Anonymous Class
<code>() -> 42</code>	<code>new Supplier<Integer>() { public Integer get() {return 42; } }</code>
<code>(x, y) -> x + y</code>	<code>new BinaryOperator<Integer>() { public Integer</code>

	<code>apply(Integer x, Integer y) { return x + y; } }</code>
<code>s -> s.length()</code>	<code>new Function<String, Integer>() { public Integer apply(String s) { return s.length(); } }</code>
<code>(String s) -> {System.out.println(s); }</code>	<code>new Consumer<String>() { public void accept(String s) { System.out.println(s); } }</code>

1.4 Variable Capture

- Lambdas can access:
 - **Instance variables** (from enclosing class) – allowed.
 - **Static variables** – allowed.
 - **Local variables** – only if they are **effectively final** (not modified after initialization).
- This is similar to anonymous inner classes (Java 8+ relaxed the `final` requirement to "effectively final").

1.5 Type Inference

- Java compiler infers the parameter types from the **functional interface** context.
- `Comparator<String> comp = (s1, s2) -> s1.compareTo(s2);` – types inferred as String.

1.6 When to Use Lambdas?

- Implementing functional interfaces (single abstract method).
- Passing behavior as data (e.g., sorting, filtering, event handling).
- Replacing verbose anonymous inner classes.

2. Functional Interfaces

2.1 Definition

- An interface with **exactly one abstract method** (SAM – Single Abstract Method).
- Can have default and static methods (Java 8).
- `@FunctionalInterface` annotation is optional but recommended (compiler checks).

2.2 Important Built-in Functional Interfaces (java.util.function)

Interface	Abstract Method	Signature	Usage
<code>Predicate<T></code>	<code>test(T t)</code>	<code>boolean test(T t)</code>	Condition testing (filter)

<code>Consumer<T></code>	<code>accept(T t)</code>	<code>void accept(T t)</code>	Operation with side effect (forEach)
<code>Function<T,R></code>	<code>apply(T t)</code>	<code>R apply(T t)</code>	Transformation (map)
<code>Supplier<T></code>	<code>get()</code>	<code>T get()</code>	Supply values (no input)
<code>UnaryOperator<T></code>	<code>apply(T t)</code>	<code>T apply(T t)</code>	Function where input=output
<code>BinaryOperator<T></code>	<code>apply(T t1, T t2)</code>	<code>T apply(T t1, T t2)</code>	Combine two values

2.3 Primitive Specializations

- `IntPredicate`, `LongPredicate`, `DoublePredicate` – avoid auto-boxing.
- `IntFunction<R>`, `ToIntFunction<T>` – primitive-specific.

2.4 Bi-Variants

- `BiPredicate<T,U>` – two arguments → boolean.
- `BiConsumer<T,U>` – two arguments → void.
- `BiFunction<T,U,R>` – two arguments → result.

2.5 Example Usage

```
Predicate<Integer> isEven = n -> n % 2 == 0;
Consumer<String> printer = System.out::println;
Function<String, Integer> length = String::length;
Supplier<Double> random = Math::random;
```

3. Stream API

3.1 What is a Stream?

- A sequence of elements supporting **functional-style operations** on collections.
- **Not a data structure** – it doesn't store data.
- **Pipelined** – operations are chained (lazy evaluation for intermediate ops).
- **Internal iteration** – the stream handles iteration, not the developer.

3.2 Stream vs Collections

Aspect	Collections	Streams
--------	-------------	---------

Aspect	Collection	Stream
Storage	Stores data	Does not store (pipeline)
Modification	Can add/remove elements	Cannot modify source
Iteration	External (for-each)	Internal (stream handles)
Reusability	Can be traversed multiple times	Can be consumed only once
Computation	Eager	Lazy (intermediate ops) + Terminal (eager)

3.3 Creating Streams

Source	Code
From Collection	<code>list.stream()</code> or <code>list.parallelStream()</code>
From Array	<code>Arrays.stream(array)</code>
From values	<code>Stream.of(1, 2, 3)</code>
From file lines	<code>Files.lines(Path)</code>
Infinite stream	<code>Stream.iterate(0, n -> n+1)</code> or <code>Stream.generate(Math::random)</code>

3.4 Intermediate Operations (Lazy)

Operation	Description	Example
<code>filter(Predicate)</code>	Selects elements	<code>filter(n -> n > 5)</code>
<code>map(Function)</code>	Transforms each element	<code>map(s -> s.length())</code>
<code>flatMap(Function)</code>	Flattens nested collections	<code>flatMap(list -> list.stream())</code>
<code>distinct()</code>	Removes duplicates	<code>distinct()</code>
<code>sorted()</code>	Sorts (natural or comparator)	<code>sorted(Comparator.reverseOrder())</code>
<code>limit(n)</code>	Truncates to n elements	<code>limit(10)</code>
<code>skip(n)</code>	Skips first n elements	<code>skip(5)</code>
<code>peek(Consumer)</code>	For debugging (side effect)	<code>peek(System.out::println)</code>

3.5 Terminal Operations (Eager)

Operation	Description	Example

<code>forEach(Consumer)</code>	Performs action for each	<code>forEach(System.out::println)</code>
<code>collect(Collector)</code>	Accumulates into collection	<code>collect(Collectors.toList())</code>
<code>toList()</code> (Java 16+)	Collects to list	<code>toList()</code>
<code>toArray()</code>	Converts to array	<code>toArray(String[]::new)</code>
<code>reduce(BinaryOperator)</code>	Combines elements	<code>reduce((a,b) -> a + b)</code>
<code>count()</code>	Returns number of elements	<code>count()</code>
<code>anyMatch(Predicate)</code>	Checks if any match	<code>anyMatch(n -> n > 10)</code>
<code>allMatch(Predicate)</code>	Checks if all match	<code>allMatch(n -> n > 0)</code>
<code>noneMatch(Predicate)</code>	Checks if none match	<code>noneMatch(n -> n < 0)</code>
<code>findFirst()</code>	Returns first element (Optional)	<code>findFirst()</code>
<code>findAny()</code>	Returns any element	<code>findAny()</code>
<code>min(Comparator)</code> / <code>max(Comparator)</code>	Extremum	<code>max(Comparator.naturalOrder())</code>

3.6 Collectors (java.util.stream.Collectors)

Collector	Description
<code>toList()</code> / <code>toSet()</code>	Collect to list/set
<code>toMap(keyMapper, valueMapper)</code>	Collect to map
<code>joining(delimiter)</code>	Joins strings
<code>groupingBy(Function)</code>	Groups by key
<code>partitioningBy(Predicate)</code>	Partitions into two groups
<code>counting()</code> , <code>summingInt()</code> , <code>averagingInt()</code>	Aggregations
<code>summarizingInt()</code>	Stats (count, sum, min, avg, max)

3.7 Parallel Streams

- `parallelStream()` – splits workload across multiple threads (Fork-Join pool).
- Only use when operations are **independent** and **stateless**.
- Order of processing may be non-deterministic

- Order of processing may be non-deterministic.
- Not always faster (overhead of thread management).

4. Optional Class

4.1 Why Optional?

- Avoids `NullPointerException` by explicitly representing the **presence or absence** of a value.
- Forces the developer to think about null cases.
- Provides functional-style operations for handling missing values.

4.2 Creating Optional

Method	Description
<code>Optional.of(value)</code>	Creates Optional with non-null value (throws NPE if null).
<code>Optional.ofNullable(value)</code>	Creates Optional that may be empty if value is null.
<code>Optional.empty()</code>	Creates an empty Optional.

4.3 Important Methods

Method	Description
<code>isPresent()</code>	Returns true if value exists.
<code>ifPresent(Consumer)</code>	Executes consumer if value exists.
<code>orElse(default)</code>	Returns value or default if empty.
<code>orElseGet(Supplier)</code>	Returns value or supplier-provided default.
<code>orElseThrow()</code> / <code>orElseThrow(Supplier)</code>	Returns value or throws exception.
<code>map(Function)</code>	Applies function if present; returns Optional.
<code>flatMap(Function)</code>	Similar, but function returns Optional.
<code>filter(Predicate)</code>	Returns Optional if predicate matches.

4.4 Best Practices

- **Do not** use `Optional` for fields (creates unnecessary objects).
- **Do not** use `Optional` as method parameters (creates awkward API).
- **Do** use `Optional` for return types that may be empty (e.g., `findFirst()`).
- **Do** use `orElse()` without checking `isPresent()` (default value is

- Do not call `get()` without checking `isPresent()` – defeats purpose.

4.5 Example

```
Optional<String> name = Optional.ofNullable(getName());
String display = name.map(String::toUpperCase)
                    .orElse("Unknown");
```

5. Method References

5.1 What are Method References?

- A shorthand syntax for lambdas when you're just calling an existing method.
- `ClassName::methodName` or `instance::methodName` or `ClassName::new`.

5.2 Types of Method References

Type	Syntax	Example (Lambda)	Example (Reference)
Static method	<code>Class::staticMethod</code>	<code>s -> Math.sqrt(s)</code>	<code>Math::sqrt</code>
Instance method of a specific object	<code>instance::method</code>	<code>s -> printer.print(s)</code>	<code>printer::print</code>
Instance method of any object of a type	<code>Class::instanceMethod</code>	<code>s -> s.length()</code>	<code>String::length</code>
Constructor	<code>Class::new</code>	<code>() -> new ArrayList()</code>	<code>ArrayList::new</code>

5.3 Examples

```
// Static method reference
Function<Double, Double> sqrt = Math::sqrt;

// Instance method of specific object
List<String> list = new ArrayList<>();
Consumer<String> add = list::add;

// Instance method of any object of type
Function<String, Integer> length = String::length;

// Constructor reference
Supplier<ArrayList<String>> supplier = ArrayList::new;
```

5.4 When to Use Method References?

- When lambda body is a simple method call.
 - Improves readability – code becomes more declarative.
-

6. LeetCode Problems – Approach Discussion

6.1 Merge Intervals

- **Problem:** Given array of intervals `[start, end]`, merge overlapping intervals.
- **Approach:**
 - i. Sort intervals by start time using a lambda comparator.
 - ii. Initialize merged list with first interval.
 - iii. Iterate through sorted intervals:
 - If `current.start ≤ last.end` → merge (update `last.end = max(last.end, current.end)`).
 - Else add current to list.
 - iv. Return merged intervals.
- **Time complexity:** $O(n \log n)$ due to sorting.
- **Java 8 usage:**
 - Sorting with lambda: `intervals.sort((a,b) -> Integer.compare(a[0], b[0]))`
 - `Arrays.stream()` to convert to array if needed.
 - `Collectors.toList()` for collecting results.

6.2 Binary Search

- **Problem:** Given sorted array, find target index. Return -1 if not found.
 - **Approach:** Standard iterative binary search.
 - **Time complexity:** $O(\log n)$.
 - **Java 8 usage:**
 - `Arrays.binarySearch()` already exists (but problem expects implementation).
 - Can use `Collections.binarySearch()` for lists.
 - Functional approach not directly applicable; but lambda could be used in a custom search with `IntStream.range()` (though less efficient).
-

7. HackerRank: Java Lambda Expressions

- **Task:** Given a list of integers, perform operations:

- `isOdd` – check if number is odd

- `isOdd` – check if number is odd.
- `isPrime` – check if number is prime.
- `isPalindrome` – check if number is palindrome.
- **Key concept:** Write a generic lambda expression or method that accepts a `Predicate<Integer>` and prints "ODD", "PRIME", "PALINDROME" based on evaluation.
- **Skills practiced:**
 - Defining custom functional interface (or using `Predicate`).
 - Passing lambda as argument to method.
 - Using `switch` or if-else to select the correct lambda.

8. Corporate Use Case: Real-Time Data Analytics Pipeline

8.1 Problem Context

An e-commerce company collects real-time data from various sources:

- User clicks (product ID, timestamp, action type).
- Transactions (order ID, amount, user ID, timestamp).
- Sensor data from warehouses (temperature, humidity, location).

The analytics pipeline must:

- **Filter** out invalid or irrelevant events (e.g., clicks with missing product ID).
- **Transform** raw events into structured data (e.g., map click → `ClickEvent` object).
- **Aggregate** metrics (e.g., total sales per hour, top-10 products by clicks).
- **Reduce** large datasets (e.g., compute average transaction amount).
- **Handle missing data** gracefully (e.g., if user ID missing, use "anonymous").
- **Process** in near real-time with low latency.

8.2 Application of Day 9 Topics

Requirement	Java Feature Used
Filter out invalid events	<code>Stream.filter()</code> with <code>Predicate</code>
Transform raw data to objects	<code>Stream.map()</code> with <code>Function</code>
Aggregate total sales per hour	<code>Stream.collect(Collectors.groupingBy())</code> + <code>summingDouble()</code>
Top-10 products by ..	<code>groupingBy</code> + <code>counting()</code> + <code>sorted()</code> + <code>limit()</code>

clicks	
Compute average transaction amount	<code>stream.mapToDouble().average()</code>
Handle missing user ID	<code>Optional.ofNullable(event.userId).orElse("anonymous")</code>
Parallel processing for large datasets	<code>parallelStream()</code> (if independent)
Print results to dashboard	<code>forEach()</code> with <code>Consumer</code>
Join string logs for reporting	<code>Collectors.joining("\n")</code>
Convert processed results to list	<code>collect(Collectors.toList())</code>
Sort products by sales	<code>sorted(Comparator.comparingDouble(Product::getSales).reversed())</code>
Method references for clean code	<code>Product::getSales</code> , <code>System.out::println</code>
Supply timestamps for events	<code>Supplier<Long></code> for current time
Combine metrics (sum, count, avg) in one pass	<code>Collectors.summarizingDouble()</code>

8.3 Example Data Pipeline (Conceptual)

```
// Raw event source (list of maps or JSON strings)
List<Map<String, Object>> rawEvents = fetchEvents();

// Pipeline stages
Map<String, Double> hourlySales = rawEvents.stream()
    .filter(e -> e.containsKey("amount") && e.containsKey("timestamp"))
    .map(e -> new Transaction(
        (String) e.get("userId"),
        (Double) e.get("amount"),
        (Long) e.get("timestamp")
    ))
    .collect(Collectors.groupingBy(
        t -> formatHour(t.getTimestamp()), // Group by hour
        Collectors.summingDouble(Transaction::getAmount)
    ));
```

```
// Top products by clicks
List<Map.Entry<String, Long>> topProducts = clickEvents.stream()
    .filter(e -> e.getProductID() != null)
    .map(e -> e.getProductID())
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))
    .entrySet().stream()
    .sorted(Map.Entry.<String, Long>comparingByValue().reversed())
    .limit(10)
    .collect(Collectors.toList());

// Safe handling of missing user IDs
String userId = Optional.ofNullable(event.getUserId())
    .orElse("anonymous");
```

8.4 Discussion Questions

- Why use `Optional` for user ID? (Avoids null checks; forces explicit handling of missing data.)
- How to process data in parallel safely? (Ensure operations are stateless and independent; avoid shared mutable state.)
- How to debug a stream pipeline? (Use `peek(System.out::println)` to inspect intermediate values.)
- Why use `Collectors.groupingBy` vs manual map accumulation? (Concise, less error-prone, built-in concurrency support.)

9. Hands-on Exercises (Without Code Lines)

9.1 Lambda Expression Writing

Write lambda equivalents for:

- A `Comparator<String>` that sorts by length then alphabetically.
- A `Runnable` that prints "Hello".
- A `Predicate<Integer>` that checks if number is between 10 and 20.

9.2 Stream Pipeline Description

Describe the stream pipeline for:

- Given list of strings, filter those starting with 'A', convert to uppercase, sort, collect to list.

9.3 Optional Handling Scenarios

For each scenario, describe how `Optional` helps:

- Finding a user by ID in a map (may return null).

• Creating a stream by using a map (may return null).

- Reading a property from configuration (may be missing).
- Getting the first element from a stream (maybe empty).

9.4 Method Reference Conversion

Convert these lambdas to method references:

- `s -> s.toUpperCase()`
- `() -> new HashMap<>()`
- `(s1, s2) -> s1.compareTo(s2)`
- `x -> System.out.println(x)`

9.5 Collectors in Practice

What collector would you use for:

- Grouping employees by department.
- Computing total salary per department.
- Joining all product names with comma.
- Partitioning transactions into high-value (> 1000) and low-value.

10. Quiz & Quick Check (Conceptual)

1. Which of the following is a valid lambda expression?

- (a) `(int x, int y) -> return x+y;`
- (b) `(x, y) -> x + y`
- (c) `x -> { return x*x; }`
- (d) Both b and c

2. Which functional interface has a method that returns `boolean` ?

- (a) `Consumer`
- (b) `Supplier`
- (c) `Predicate`
- (d) `Function`

3. What is the result of `Stream.of(1,2,3).filter(n -> n>1).map(n -> n*2).count()` ?

- (a) 2
- (b) 3
- (c) 6

- (d) 4

4. Which method is used to create an empty `Optional` ?

- (a) `Optional.empty()`
- (b) `Optional.of(null)`
- (c) `Optional.ofNullable(null)`
- (d) Both a and c

5. What does `String::toUpperCase` represent?

- (a) Static method reference
- (b) Instance method reference of a specific object
- (c) Instance method reference of any object of type `String`
- (d) Constructor reference

Answers: 1-d, 2-c, 3-a (filter keeps 2 elements, count=2), 4-a (c also works but creates empty via `ofNullable`), 5-c

11. Additional Resources & Cheat Sheets

- **Lambda syntax cheat sheet:** `(params) -> expr` or `(params) -> { statements; }`
- **Functional interfaces quick reference:** `Predicate`, `Consumer`, `Function`, `Supplier`.
- **Stream operations categorized:** Intermediate vs Terminal, Lazy vs Eager.
- **Optional best practices** – when to use and when to avoid.
- **Method reference cheat sheet:**
 - Static: `Class::staticMethod`
 - Instance (specific): `instance::method`
 - Instance (any): `Class::instanceMethod`
 - Constructor: `Class::new`

12. Closing Summary

Day 9 introduced the transformative features of Java 8 that revolutionized Java programming. Lambda expressions enabled concise functional-style code, while functional interfaces provided a framework for passing behavior. The Stream API allowed declarative, pipelined processing of collections, reducing boilerplate and improving readability. `Optional` eliminated null pointer exceptions by forcing explicit handling of absent values. Method references further simplified lambda expressions. The Real-Time Data Analytics Pipeline case study demonstrated how these features combine to build clean, efficient data processing

pipelines: filtering, transforming, aggregating, and reducing data streams. LeetCode problems reinforced functional thinking, and HackerRank provided hands-on lambda practice.

Key Takeaways:

- Lambdas make code concise and expressive.
 - Built-in functional interfaces cover common patterns.
 - Streams enable declarative collection processing with internal iteration.
 - Optional prevents null-related bugs.
 - Method references clean up repetitive lambdas.
-

End of Day 9